

20200622面试题答案解析

```
var urls = [  
  'http://jsonplaceholder.typicode.com/posts/1',  
  'http://jsonplaceholder.typicode.com/posts/2',  
  'http://jsonplaceholder.typicode.com/posts/3',  
  'http://jsonplaceholder.typicode.com/posts/4',  
  'http://jsonplaceholder.typicode.com/posts/5',  
  'http://jsonplaceholder.typicode.com/posts/6',  
  'http://jsonplaceholder.typicode.com/posts/7',  
  'http://jsonplaceholder.typicode.com/posts/8',  
  'http://jsonplaceholder.typicode.com/posts/9',  
  'http://jsonplaceholder.typicode.com/posts/10'  
]  
  
function loadDate (url) {  
  return new Promise((resolve, reject) => {  
    const xhr = new XMLHttpRequest()  
    xhr.onload = function () {  
      resolve(xhr.responseText)  
    }  
    xhr.open('GET', url)  
    xhr.send()  
  })  
}
```

在 urls 数组中存放了 10 个接口地址。同时还定义了一个 loadDate 函数，这个函数接受一个 url 参数，返回一个 Promise 对象，该 Promise 在接口调用成功时返回 resolve，失败时返回 reject。

要求：任意时刻，同时下载的连接数量不可以超过 3 个。

试写出一段代码实现这个需求，要求尽可能快速地将所有接口中的数据得到。

解题思路

按照题意我们可以这样做，首先并发请求 3 个 url 中的数据，当其中一条 url 请求得到数据后，立即发起对一条新 url 上数据的请求，我们要始终让并发数保持在 3 个，直到所有需要加载数据的 url 全部都完成请求并得到数据。

用 Promise 实现的思路就是，首先并发请求 3 个 url，得到 3 个 Promise，然后组成一个叫 promises 的数组。再不断的调用 Promise.race 来返回最快改变状态的 Promise，然后从数组 promises 中删掉这个 Promise 对象，再加入一个新的 Promise，直到所有的 url 被取完，最后再使用 Promise.all 来处理一遍数组 promises 中没有改变状态的 Promise。

参考答案

```
var urls = [
  'http://jsonplaceholder.typicode.com/posts/1',
  'http://jsonplaceholder.typicode.com/posts/2',
  'http://jsonplaceholder.typicode.com/posts/3',
  'http://jsonplaceholder.typicode.com/posts/4',
  'http://jsonplaceholder.typicode.com/posts/5',
  'http://jsonplaceholder.typicode.com/posts/6',
  'http://jsonplaceholder.typicode.com/posts/7',
  'http://jsonplaceholder.typicode.com/posts/8',
  'http://jsonplaceholder.typicode.com/posts/9',
  'http://jsonplaceholder.typicode.com/posts/10'
]

function loadDate (url) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest()
    xhr.onload = function () {
      resolve(xhr.responseText)
    }
    xhr.open('GET', url)
    xhr.send()
  })
}

function limitLoad(urls, handler, limit) {
  // 对数组进行一个拷贝
  const sequence = [].concat(urls)
  let promises = []

  //实现并发请求达到最大值
  promises = sequence.splice(0, limit).map((url, index) => {
    // 这里返回的 index 是任务在数组 promises 的脚标
    //用于在 Promise.race 后找到完成的任务脚标
    return handler(url).then(() => {
      return index
    });
  });

  // 利用数组的 reduce 方法来以队列的形式执行
  return sequence.reduce((last, url, currentIndex) => {
    return last.then(() => {
      // 返回最快改变状态的 Promise
      return Promise.race(promises)
    }).catch(err => {
      // 这里的 catch 不仅用来捕获前面 then 方法抛出的错误
      // 更重要的是防止中断整个链式调用
      console.error(err)
    }).then((res) => {
      // 用新的 Promise 替换掉最快改变状态的 Promise
      promises[res] = handler(sequence[currentIndex]).then(
        () => { return res });
    })
  }, Promise.resolve()).then(() => {
    return Promise.all(promises)
  })
}

limitLoad(urls, loadDate, 3)
```

```
/*  
因为 loadDate 函数也返回一个 Promise  
所以当 所有图片加载完成后可以继续链式调用  
  
limitLoad(urls, loadDate, 3).then(() => {  
    console.log('所有url数据请求成功');  
}).catch(err => {  
    console.error(err);  
})  
*/
```